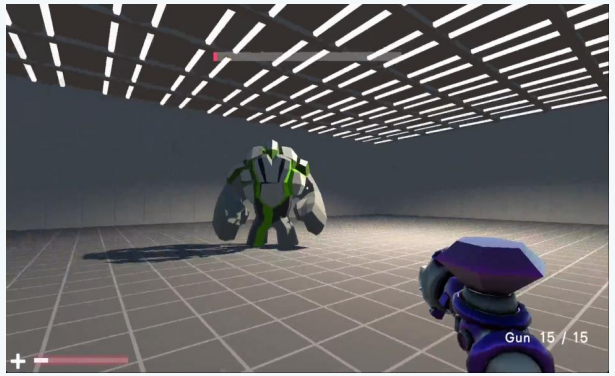
プロジェクト概要

injong95j@gmail.com



クライアントプログラマー

CHOI INJONG (チェ・インジョン)

The X

項目	内容
プロジェクト名	The X
ジャンル	推理アクション
開発人数 / 期間	1名（個人開発） / 約4ヶ月
プラットフォーム	Windows
開発環境	Unity 6.2 / Visual Studio 2022
外部ライブラリ	Newton.Json/ Outline
ゲームの目的	クルーに寄生した謎の生命体「X」を特定し排除する

実行ファイル

ソースコード

プレイ映像

このゲームは…

手がかりを収集し…

クルーたちを観察して謎の生命体「X」を特定し…

謎の生命体「X」を倒すゲームです。

Part 1

プログラム制作において工夫した点

1.1 役割を分離し、結合度を下げた構造の構築

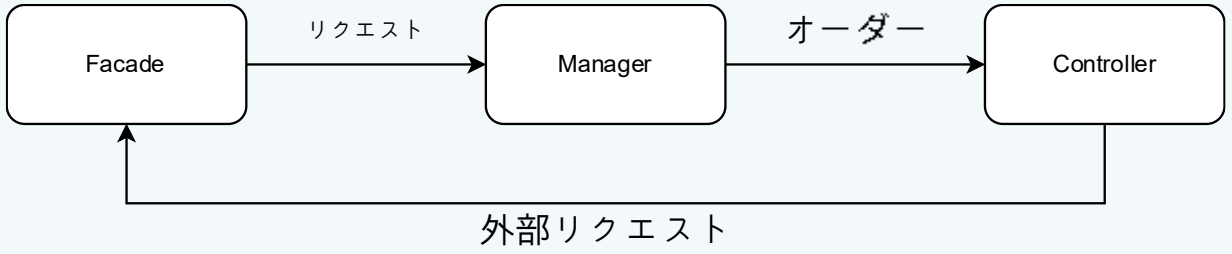
本プロジェクトで一番 力を入れた 部分は、全体の「構造」です。
過去に**God Class**や**スパゲッティコード**で苦労した経験があり、それを防ぐために役割の分離と**Facade**パターンを導入しました。

役割の分離

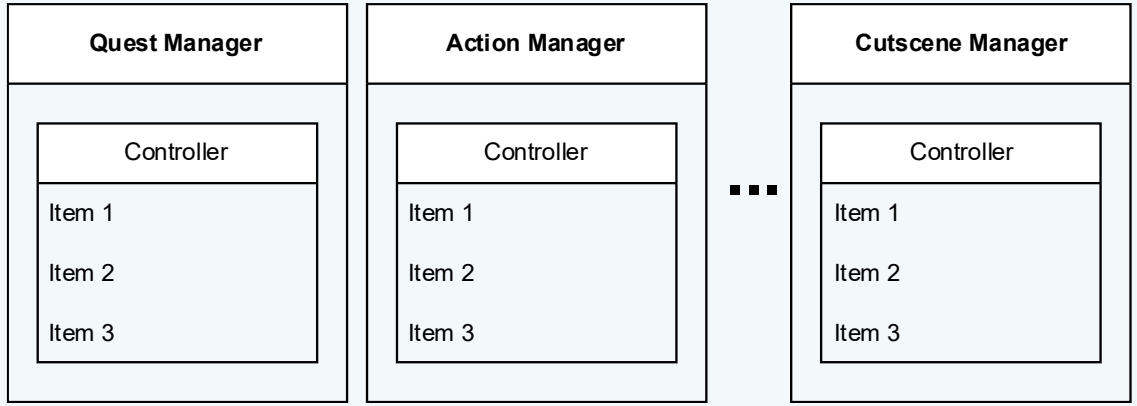
Managerクラスはデータと**Controller**の「管理」だけを行い、**Controller**は実際のロジックを担当するように役割を明確に分けました。
これによって、特定のクラスが大きくなりすぎることを防ぎました。

Facadeパターンの導入

外部からの機能リクエストは、**Facade**という一つの窓口を通すように統一しました。クラス同士が直接参照し合うことを防ぐことで、スパゲッティコードになる可能性を遮断し、システム全体の結合度を下げることができました。



プロジェクト全体の流れ



各機能ごとに複数の**Manager**と**Controller**が存在します。

Part 1

プログラム制作において工夫した点

クラス名や名前をクリックすると詳細を確認できます。

```
public void RequestMoveScene(string sceneName)
{
    GameSceneManager.Instance.TryMoveScene(sceneName);
}
public void RequestPlayCutscene(string cutsceneName)
{
    CutsceneManager.Instance.TryPlayCutscene(cutsceneName);
}
```

Facade

```
public void PlayCutscene(CutsceneData cutsceneData)
{
    CreateCutsceneObject(cutsceneData.Path);
    if (CutsceneObject == null)
        return;

    m_timlineHandler.Init(CutsceneObject);

 m_timlineHandler.SetDynamicBindings(cutsceneData.TrackDynamicBindingD
ic);
    m_timlineHandler.Play();

    RefreshCutsceneState(CutsceneStateEnum.Play);
}
```

CutsceneController

```
private Dictionary<string, CutsceneData> m_cutsceneDic;
private CutsceneController m_controller;

public void TryPlayCutscene(string cutsceneName)
{
    ...
    Facade.Instance.RequestScreenFadeIn(1f, () =>
    {
        m_controller.PlayCutscene(cutsceneData);
        m_controller.PauseCutscene();
        Facade.Instance.RequestScreenFadeOut(1f, () =>
        {
            m_controller.ResumeCutscene();
        });
    });
}
```

CutsceneManager

Part 1

プログラム制作において工夫した点

1.2 MVCパターンの導入とインターフェースによる制御

UIの変更がコアロジックに影響を与えないよう、**制御（Controller）**、**データ（Session）**、**表示（View）**を完全に分離しました。
また、インターフェースベースで機能を制御する設計により、テストと修正が容易な構造を構築しました。

MVCパターン

Model → DialogSession

ダイアログのノードデータと現在の進行状態だけを純粋に保持し、内部のロジック処理は一切行わないように構成しました。

View → UIDialog

DOTweenを活用したテキストの出力や選択肢ボタンの表示など、
視覚的な演出と出力のみを担当するように構成しました。

Controller → DialogController

Modelからデータを取得し、分岐条件（クエストの完了状況など）を判断して、次の行動を決定し、Viewに出力を指示するロジックを実装しました。

インターフェースによる制御

IDialogや**IAction**などのインターフェースを通じてオブジェクトを制御できるように実装しました。

これにより、**Controller**が内部の具体的な実装の詳細を知らなくても機能を実行できるよう設計することで保守性向上させました。

Part 1

プログラム制作において工夫した点

クラス名や名前をクリックすると詳細を確認できます。

```
private IDialog m_dialogUI;
private DialogSession m_session;

private void SetDialogData()
{
    m_currentNode = m_session.GetNodeToSetup();
    m_dialogUI.SetData(m_currentNode, ShowTargetDialog);
    SetDialogNode(m_currentNode);
}

public void StartDialog(DialogData data)
{
    ...

    m_session.SetSession(data.StartNodeID, data.Nodes);
        ...
    SetDialogData();
    PlaySound();
    m_dialogUI.ShowDialog();


 m_session.RefreshSetUpNode(GetRealNextNodeID(m_currentNode));
        ...
}
```

DialogController

```
private Dictionary<string, DialogNode> m_nodeDic;
private DialogNode m_setUpNode;

public void SetSession(string startNodeID, Dictionary<string,
DialogNode> nodeDic)
{
    m_nodeDic = new Dictionary<string, DialogNode>(nodeDic);
    m_setUpNode = m_nodeDic[startNodeID];
}

public DialogNode GetTargetNode(string targetID)
{
    return targetID == null ? m_setUpNode : m_nodeDic[targetID];
}
```

DialogSession

Part 1

プログラム制作において工夫した点

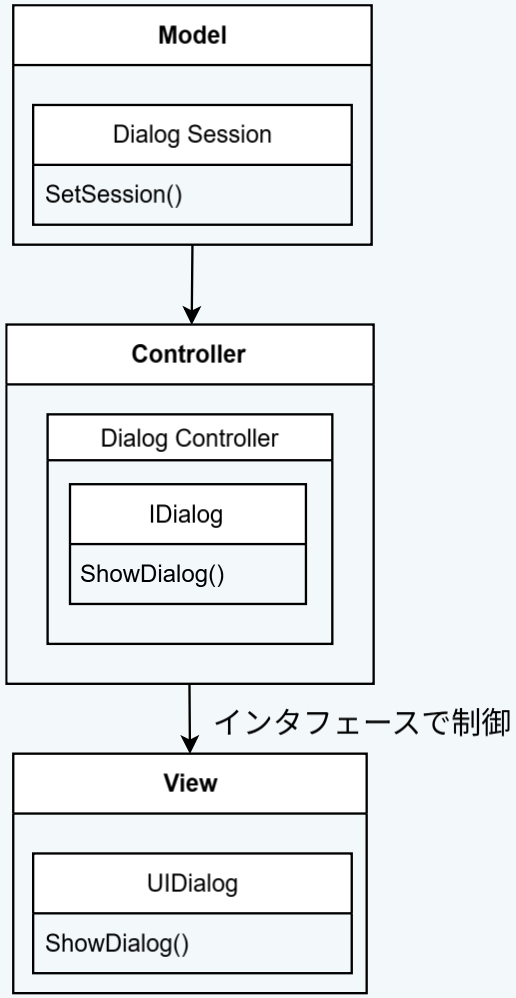
クラス名や名前をクリックすると詳細を確認できます。

```
public interface IDialog
{
    public void SetData(DialogNode node, Action<string>
buttonAction = null);
    public void ShowDialog();
    public void HideDialog();
    public bool IsProgressing();
    public void EndThisDialog();
}
```

IDialog

```
public void ShowDialog()
{
    Show();
    ...
    m_dialogText.DOKill();
    m_dialogText.text = "";
    m_dialogText.DOText(m_currentText, m_currentText.Length * 0.2f);
}
```

UIDialog



コントローラーはインタフェースでUIを制御します。

Part 2

プログラムを作成する上で大変だった所、そして解決

直面した課題

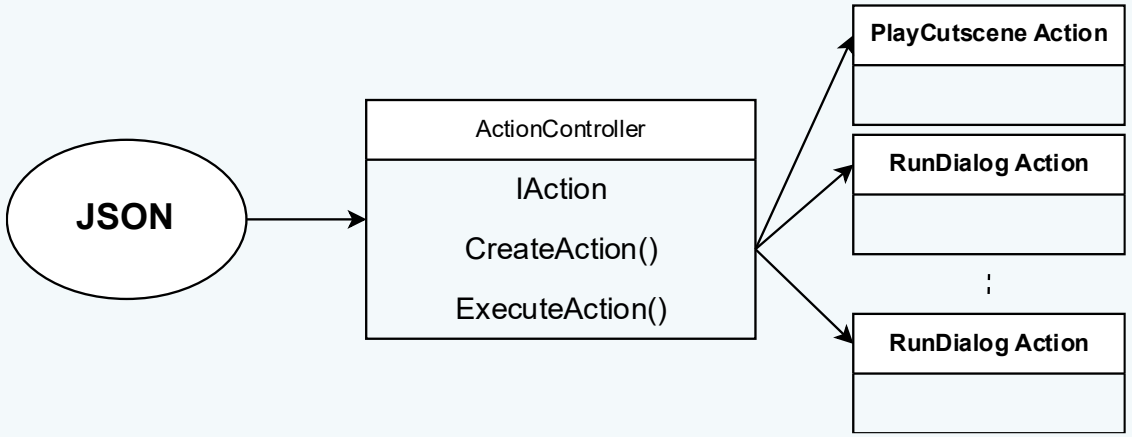
特定状況の実装におけるハードコーディング問題と拡張性の低下

観察、会話、シーン移動など、状況別のインタラクションが追加されるたびにハードコーディングが発生し、システムの拡張性が大きく低下する問題に直面しました。

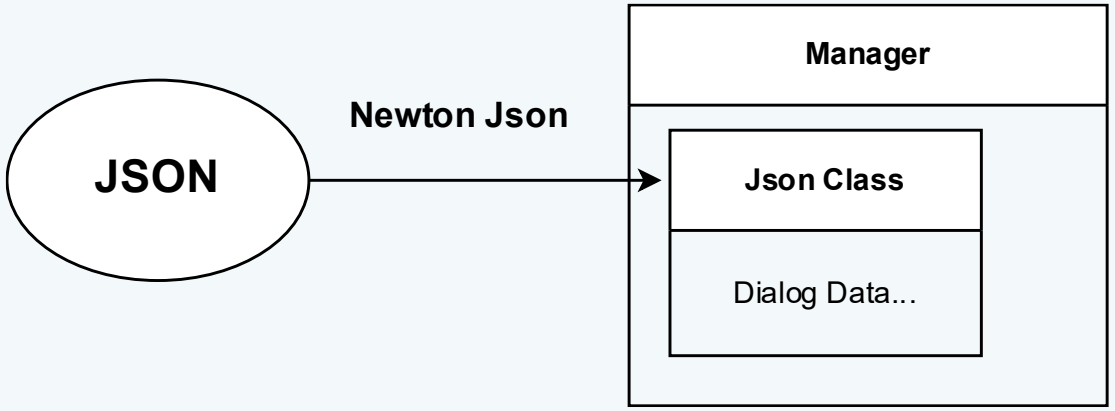
解決策

Data-Drivenの構築

- 外部からコードを修正することなくゲームの進行を自由に制御できるよう、「状況」と「アクション」を分離しました。
- ゲーム内で発生する最小単位のアクションを、**IAction**インターフェースをベースにモジュール化しました。
- 特定の状況で実行されるべきアクションのリストをJSONデータとして定義しました。
- 結果として、まるでレゴブロックを組み立てるように、外部のJSONデータ（**ActionInfo**）の組み合わせだけで、コードの修正や再コンパイルを行うことなく、多様な演出やロジックの流れを制御できるシステムを構築しました。



外部からアクションを制御できるようにすることで、
拡張性を大幅に向上させました。



パースされたJSONデータは
JsonClassに変換されて使用されます。

Part 2

プログラムを作成する上で大変だった所、そして解決

クラス名や名前をクリックすると詳細を確認できます。

```
{
  "actionInfo":
  [
      ...
    "MoveScene",
    "PlayCutScene",
    "RunDialog",
    "RunJudgement",
    "RunSave",
  ]
}
```

ActionInfo.json

```
{
  "startNodeID": "ReturnMachine_Main",
  "nodes":
  {
      ...
    "Action_Return":
    {
      "talker": "観察室システム",
      "text": "",
      "action":
      {
        "actionType": "MoveScene",
        "targetID": "Stage1"
      }
    }
  }
}
```

TeleportMachine.json

Part 2

プログラムを作成する上で大変だった所、そして解決

クラス名や名前をクリックすると詳細を確認できます。

```
private IAction m_action;

public void ExecuteAction(ActionData actionData)
{
    m_action = CreateAction(actionData);
    ...
    m_action?.OnExecute();
}

private IAction CreateAction(ActionData actionData)
{
    IAction action = null;

    switch (actionData.ActionType)
    {
                    ...
        case ActionTypeEnum.MoveScene:
            action = new MoveSceneAction();
            break;
            ...
    }

    action?.InitAction(actionData);
    return action;
}
```

ActionController

```
public class MoveSceneAction : IAction
{
    public ActionTypeEnum ActionType { get; }
    private string m_sceneName;

    public void InitAction(ActionData actionData)
    {
        m_sceneName = actionData.TargetID;
    }

    public void OnExecute()
    {
        Facade.Instance.RequestMoveScene(m_sceneName);
    }
}
```

MoveSceneAction

Part 1

プログラム制作において工夫した点

クラス名や名前をクリックすると詳細を確認できます。

```json
{
  "id": "Q_CollectAllClue",
  "questType": "Clue",
  "title": "手がかり収集",
  "description": "手がかりは至る所にある。",
  "targetList": [ "C_BigSpot", "C_BlueColor", "C_ElbowLocation"],
  "completeAction":
  {
    "actionType": "RunDialog",
    "targetID": "CompleteClueQuest"
  }
}
```

QuestInfo.json

```csharp
[JsonObject]
public class QuestData
{
    [JsonProperty("id")]
    public string ID { get; private set; }

    [JsonProperty("questType")]
    public QuestTypeEnum QuestType { get; private set; }

    [JsonProperty("title")]
    public string Title { get; private set; }

    [JsonProperty("description")]
    public string Description { get; private set; }

      ...

    [JsonProperty("completeAction")]
    public ActionData CompleteActionData { get; private set; }
}
```

JsonClass

```csharp
public string Title { get; private set; }
public string Description { get; private set; }
public string TargetID { get; private set; }

public void InitQuest(QuestData questData)
{
    ...
    TargetHash = new HashSet<string>();
    TargetHash = questData.TargetHash;

    GoalCount = TargetHash.Count;
    ...
    ClueManager.OnUpdateClueAction += HandleUpdateClueAction;
}

public void UpdateQuest()
{
    if (CurrentCount >= GoalCount)
    {
        CompleteQuest();
        return;
    }

    OnUpdateQuestAction?.Invoke();
}
```

ClueQuest

Part 3

特に注意してみてもらいたい所

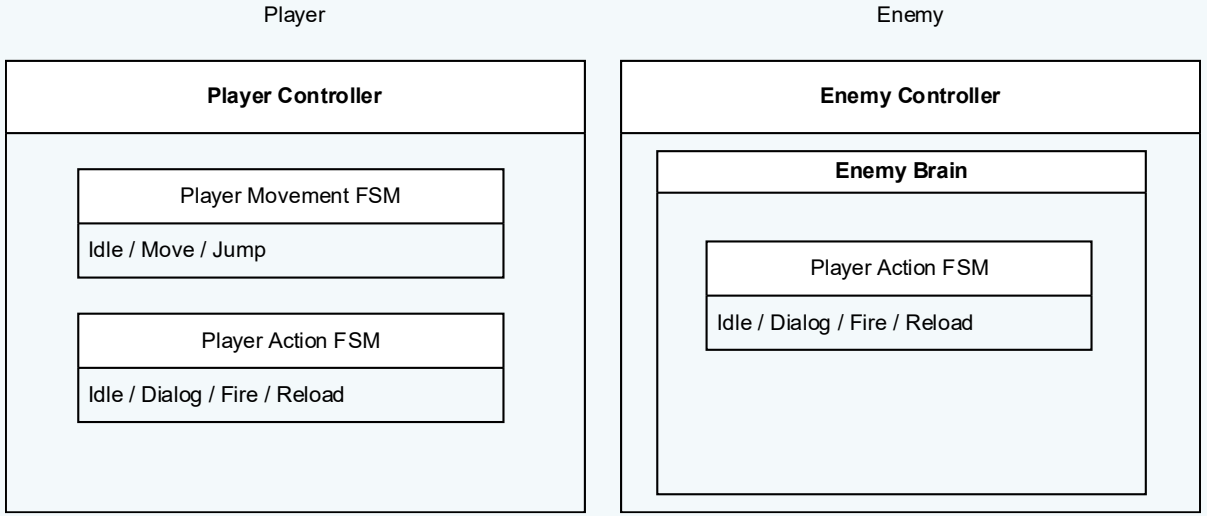
FSMの導入により可読性と保守性を向上させ、**EnemyBrain**を実装してロジックと判断を分離することで、データ中心の柔軟な拡張実現しました。

FSMとメモリ最適化

- FSMを適用し、各行動をクラスとして分離することで、巨大な条件分岐による可読性の低下を防ぎました。
- また、状態遷移時に発生する**GC**の負荷を防ぐため、初期化の時点で状態オブジェクトを**Dictionary**にキャッシュし、メモリパフォーマンスを最適化しました。

ロジックと判断の分離：Enemy Brain システム

- 敵の実際の「行動」は**FSM**が実行しますが、次の行動に対する「判断」は**EnemyBrain**クラスが独立して担当するように実装しました。
- **EnemyBrain**は外部JSON（**EnemyInfo**）から注入されたデータを基に次の行動を計算し、**FSM**に状態遷移を指示します。
- これにより、多様な状況に柔軟に対応できるようにしました。



Playerと**Enemy**は、**FSM**によって状態が制御されます。

Part 3

特に注意してみてもらいたい所

クラス名や名前をクリックすると詳細を確認できます。

```
private PlayerMovementFSM m_movementFSM;
private PlayerActionFSM m_actionFSM;

private void InitFSM()
{
    m_movementFSM = new PlayerMovementFSM(this);
    m_actionFSM = new PlayerActionFSM(this);
}

private void Update()
{
    m_movementFSM.Update();
    m_actionFSM.Update();

    m_timer -= Time.deltaTime;
    if (m_timer <= 0f)
    {
        m_timer = m_interactInterval;
        CheckInteractable();
    }
}
```

PlayerController

```
private Dictionary<PlayerStateEnum, IState> m_stateDic;

public PlayerMovementFSM(PlayerController playerController)
{
        ...
    m_stateDic.Add(PlayerStateEnum.MovementIdle, new
PlayerMovementIdleState(this, playerController));
        ...
    ChangeState(PlayerStateEnum.MovementIdle);
}

public void ChangeState(PlayerStateEnum newState)
{
    CurrentState?.Exit();
    CurrentState = m_stateDic[newState];
    CurrentStateEnum = newState;
    CurrentState.Enter();
}

public void Update()
{
    CurrentState?.Update();
}
```

PlayerMovementFSM

特に注意してみてもらいたい所

クラス名や名前をクリックすると詳細を確認できます。

```
{
    "EnemyInfo":
    {
        "Astronaut_Alien_Enemy":
        {
            "HP": 500,
            "Attack": 10,
            ...
            "Action": [ "Move", "Skill" ],
            "Skill": [ "ShockWaveAttack", "RushAttack"],
            ...
        }
    }
}
```

EnemyInfo

```
private EnemyFSM m_fsm;
public float RandomActionInterval { get; private set; }


private void ExecuteRandomAction()
{
    int randomIndex = Random.Range(0, m_actionList.Count);
    EnemyActionEnum action = m_actionList[randomIndex];

    var actionState = m_fsm.CurrentState as IEnemyActionState;
    switch (action)
    {
        case EnemyActionEnum.Idle:
            actionState?.HandleIdle();
            break;
        ...
    }
}

public void ExecuteSkill(Action callback)
{
    m_skillHandler.ExecuteSkill(callback);
}

private void SetRandomSkill()
{
    ...
    int randomIndex = Random.Range(0, SkillList.Count);
    SkillEnum skill = SkillList[randomIndex];

    m_skillHandler.InitSkill(skill);
}
```

EnemyBrain